

Demonstration Variant of the ICEF ICS & Python Programming Midterm Exam

I. Excel

Task 1.

You are given data on the videogame sales in the form of an excel book. You must copy this data to a separate sheet inside your own excel book (which you'll be submitting as a part of the exam) and then **create a new list for each of the following subsections.**

1. Categorize each entry (row) in the database by 2 groups of categories: assign category "Modern" if the game was released after 2014 (including this year), "Classic" if the game was released between year 2000 (this year included) and 2014 and "OG" if the game was released before 2000, assign category "Popular" if the total sales exceed average and "Below average" if the total sales are below average. In a separate cell give the average sales of the games you'd call "Popular Classic" by the above classification method.
2. Find the scalar, by which all of the sales should be multiplied so that their sum is equal to $\sqrt{3x \sin(\beta x) + 3}$, where x is the scalar and $\beta=1.5$. Separately, create a what-if table, which shows the absolute difference between the scaled sum of all the sales and the result of the expression if x assumes the values of 0, 1, 10 and 100 and β assumes the values of -1, 5, 15 and 30.
3. Create a pivot table of the database, where the columns "Name" and "Publisher" will be used as filters, values of column "Genre" must be along the columns and the values of column "Platform" must be along the rows. In a separate cell on the same list put the total sales for PC shooters in EU plus the total sales of PSP racing games Globally

II. Python

Task 1.

Create a function and call it **filter**, which takes two arguments: *char_list*- a list of characters and *filename*- a string variable. The function should read the contents of the file with the name defined by the variable *filename* and “.txt” extension and return a list of rows where the only characters present are the ones that are **not** in the variable *char_list*. Your function is expected to handle errors related to opening the file and/ or other operations. The error is considered handled if the user receives the error message without having the function throw an error in case it occurs.

Task 2.

1. Create a function called **derivative**. It should accept two arguments: *y* and *x*, where *x* is an optional argument. Both arguments are expected to be lists. This function (as the name suggests) should compute derivatives for each point in the argument *y*, where it's possible. The function should return the list of derivatives at each point. The formula you should use for the derivative at point *y** is:

$$y'_i = \frac{y_i - y_{i-1}}{x_i - x_{i-1}}$$

By default (if the argument *x* isn't passed by the user) the denominator should be considered 1. Also note that the derivative via this formula can't be computed in the first entry of the argument *y*. The function should check that each element in each of the passed arrays is of a numeric type (int or float). If the passed argument isn't a list and can't be interpreted as one, the function should return None.

2. Create a function called **writer**. It should accept 2 arguments: *filename* of type string and *lines* of type list. The function should write the contents of the variable *lines* into the file with the name from the variable *filename*. If the contents aren't string variables,

the function should convert the contents to string. If the file doesn't exist, the function should print an error message. If the file doesn't exist, the function should print out a corresponding message. If the writing proceeds successfully, the function should print out a message. The function shouldn't return anything in any case.

Task 3.

1. Create a function named **p_detect**, which takes one argument *x* of type integer. It should return True if *x* is a palindrome, and False otherwise. A number is a palindrome if it's read the same forwards and backwards.
2. Create a function called **merge**, which takes two arguments: *arr_1* and *arr_2*, both are sorted lists. The function should return a single merged sorted list.